CAUSAL MAP GARDEN

Task 2 – Coding with AI

Contents

Intro – auto-coding with AI

AI coding overview

Coding and recoding – Dealing with overlapping labels

Intro – auto-coding with AI

Chapter contents.

15 Oct 2025

> Causal coding is fascinating but can take a lot of time. Using AI to help you is pretty easy, especially if you provide a codebook of factor labels which the AI has to use, as we will explain in this chapter, when we've finished it!

Using a pre-defined codebook is simple. If you instead let the AI invent its own labels, you have to worry about how to combine all the hundreds or even thousands of labels which it might create. This chapter also looks at what happens when you do *not* provide a codebook.

PAGES IN THIS CHAPTER

AI coding overview

Opposites and sentiment in AI coding

Coding and recoding – Dealing with overlapping labels

AI coding overview

2 May 2026

Summary

This page is an overview of the choices you face when doing AI coding (qualitative causal coding with AI) inside the Causal Map app.

- For step-by-step app docs see AI coding
- For the AI Answers panel (asking questions of coded or uncoded text) see AI answers panel
- For the broader argument about why we use AI as a low-level coder rather than as a black box, see AI in evaluation actually show your working! and Just add rigour Three do's and don'ts.

Workflow

Someone arrives with a stack of documents and asks: can you code these? It depends first on various parallel setup decisions. After coding, you face a separate decision about how, or whether, to recode the labels.

Decisions before coding

Codebook strategy: how free?

You can start with a full codebook, a partial codebook, or nothing.

A *forced* codebook restricts the model to your labels (for example from a theory of change). If a causal claim mentions a factor not in the codebook, no link is coded.

Most often you provide a codebook but allow the model to invent labels when nothing fits. This is harder to manage: telling the model it *can* improvise seems to confuse it a bit, and codebook coverage drops.

Four codebook strategies:

1. Stick only to the codebook. Anything that doesn't match is dropped.
2. Stick to the codebook mostly, but let the AI code other things too. Tell it to flag the new ones with a tag like `[new]` or a trailing `*`, so you can find and review them later.
3. Compromise. Use only top-level labels from the codebook, but let the AI improvise the second part of a hierarchical label. See Hierarchical coding.]
4. Free coding

Related to the above...

Label style: In vivo or abstract

When free coding, you can ask for *in vivo* labels (close to the original text) or for more abstract labels. There are many ways to instruct: "talk like a social scientist" or "talk like a local newspaper editor".

Label style: Using tags

There's often more to a coding task than just labels. *Tags* are short bits of text in brackets attached to labels: `stressed patients (before surgery)`, `stressed patients (after surgery)`. Tags help you build up a system of labels from smaller parts.

Label style: Opposites

See !Opposites and sentiment in AI coding and Opposites.

Label style: Hierarchical

You can impose hierarchical labels even when free coding. See Hierarchical coding.

Columns

We also often ask for *columns* unrelated to the labels, such as a sentiment assessment. Columns can be useful for any systematic attribute you can code consistently across most links. Note the difference: columns code attributes of *links*; tags code attributes of *factors*.

Asking the model to fill extra columns alongside coding is extra work for it. Expect either coverage (links found) or precision to drop.

Columns: Sentiment

Zero-codebook coding usually leads on to magnetic soft recoding, and in embedding space `decrease in X` sits close to `increase in X`.

With an explicit codebook, you can get round this by (maybe) suggesting both sides of any factor likely to appear positively and negatively, e.g. `increased income` and `decreased income`.

When you don't specify a codebook, get the model to code sentiment for each link.

Sentiment is a kind of column.

A sentiment column lets you tell them apart.

Sentiment can also be interesting for other reasons.

Other things which are important in the prompt

Often you want to tell the model what the work is for: the context, named entities, the audience. We *iterate* on the prompt, trying versions and comparing results.

Coding style: holistic or claim-by-claim?

Two main approaches.

Holistic: we ask the model for a Mermaid diagram of the causal network, then convert it into a links table. The model decides what the main links are, but we still ask for a quote behind each one. This may handle longer sources better, but we don't really know.

Claim by claim: we ask the model to find each individual causal link. Hundreds of tweaks and heuristics later, this style still struggles to tell a connected story even when the text contains one. Suppose the text supports A to B to C to D, but the model lazily codes A to B and C to D, using slightly different labels for B and C. Claim-by-claim coding only really works if you plan to recode afterwards: you hope a later recoding pass spots that B and C mean the same thing and rejoins the chain.

Model

For relatively straightforward cases, newer or bigger models are not necessarily better. We have had good results with Gemini Flash, for example.

How many iterations?

Additional iterations can be useful:

- For checking and improving quality
- For increasing coverage (finding more actual causal claims)
- For adding additional information (e.g. more columns)

Multi-stage prompts (separated by `====`) let you split coding into sequential passes. The UI does this for you. Mechanics in !1010-Auto-coding and AI coding under "Prompt sections".

Chunk and sample strategy

If a single source is short, you can map the whole thing in one go. More often you have either much longer sources, which the app breaks into chunks for you, or many sources.

Recall and precision

A pretty hard rule: the more text you give the model at once, the less dense its coding gets. One page might yield 20 links; 5 pages might also yield 20 links. Sometimes the 20 it picks out of 5 pages really are the most important, but this is not certain, and you are leaving the model to decide. Often you just want better recall, and that means smaller chunks. Don't give the model too much freedom to decide what counts as important.

What does it select

Bigger chunks mean sparser coding. There is a "code everything" setting that does not each source at all: the effect depends on how long your sources are. Otherwise, work in smaller chunks. Aiming to pick up *every* causal claim is unrealistic, but the smaller the chunk, the more you'll catch.

Iterative coding strategy with bigger corpora

If you have more than around 100 pages, work on a sample first. The app has features to take a random sample of sources, or a stratified random sample within source groups. The AI coding panel also offers a sample-only run (see AI coding). Try the sample, review, adjust your strategy, perhaps update the codebook, then run a larger sample. With 1000 pages you might code 100, adjust, code another 300 (deleting the first attempt), and if that looks good, finish with the remaining 700.

Recoding style

Once the first coding run exists, the next question is how to deal with overlapping labels. See Coding and recoding — Dealing with overlapping labels.

Recoding from scratch means arriving at a clean revised codebook and then recoding everything from the beginning. That's more expensive and slower, but usually gives better results than magnetically recoded labels alone (see Coding and recoding — Dealing with overlapping labels).

- *Hard recoding*: revise the codebook and code again.
- *Links recoding*: use AI Answers / Links to recode links, with quote and context available.
- *Factors recoding*: use AI Answers / Factors to recode factor labels directly.
- *Soft recoding*: use clustering or magnetic labels as a softer recoding layer.

Related

- chapter intro
- AI coding: app docs for the AI Coding panel
- AI answers panel: app docs for AI Answers
- !1010- Auto-coding: older detailed notes on the auto-coding prompt
- !Opposites and sentiment in AI coding
- Coding and recoding — Dealing with overlapping labels
- ! Autoclustering with embeddings
-
- AI in evaluation actually show your working!: the transparency argument
- Just add rigour Three do's and don'ts: do's and don'ts for AI text analysis
- Causal mapping is easy to automate transparently, so is a great fit for scaling with AI

🌻 Coding and recoding – Dealing with overlapping labels

So assuming you're not coded with a fixed, coding with a fixed codebook, and assuming you're putting in enough text that it's either coding multiple sources at a time or it's, sorry it's coding multiple sources or it's you've got one or more texts which are so long that you're breaking them up into multiple chunks, then it's normal that you'll have a codebook with very many labels, many of which overlap in meaning and so you've got the problem that we have discussed a fair amount that you can address either with soft recoding or magnetic relabelling, which we've done a lot, but we're also moving more nowadays to hard recoding where once we've done a fair amount of coding, or all the coding, we then group those labels either pre-clustering them using embeddings or and or applying an AI and or putting a human in the loop to get a list of labels for hard coding, or rather a system because you might have a smaller set of labels and additional tags or columns.

We haven't done the research to find out whether having say 10 labels and three tags is more or less efficient than the corresponding set of 60 labels.

And of course there's also the newer possibility of using the AI in the AI Answers feature to take an existing large set of coded labels and then recode them into a more compact set. This has also been discussed here

So you've basically got a lot of decisions to take as you work your way through the coding pathway. And it all depends on lots of different things like how long your text is, how many different documents you have. Oh, I didn't mention that in Causal Map we never, we do break down long source texts into smaller chunks but we never combine smaller source texts into larger chunks, so each source is always coded on its own. So if you don't have a fixed codebook then you'll always get many labels with overlapping meanings.

	Hard coding	**Hard recoding**	**Links recoding**	**Factors recoding**	**Soft recoding**
Accuracy	Highest	...	...	...	Lowest
Speed	Slowest	...	...	...	Fastest
Manual	Just code manually	Make a copy of your file, delete links and start again	Edit manually in Links table or Map, - or use	Edit manually in Factors table or Map, - or use	-

			search/replace in Links table	search/replace in Factors table - or Bulk Edit	
AI	Just code with AI, with/without a codebook	As above, or just put the switch "skip coded sources" to off	AI Answers / Links. Writes into whichever Label Set is active in the toolbar.	AI Answers / Factors. Writes into whichever Label Set is active in the toolbar.	Apply magnetic labels in Soft Recode filter

What's the point of Links and Factors recoding? What's the difference?

- Soft recoding is only as good as the underlying embedding space, and it is never perfect.
- Hard recoding can take a long time, is expensive, and does not encourage experimentation
- With Links/Factors recoding, you can:
  - Recode just the currently filtered sources/links (or all links)
  - Recode into whichever Label Set is active. Pick `default` in the Label Set widget (toolbar, below the Sources bar) to write to the permanent cause/effect labels; create a named set such as `experiment1` and the AI writes to `cause_experiment1` / `effect_experiment1` instead. Flip between sets in the same widget to view either.
  - There is also another option Answers which is not about recoding; it is simply a way to send your links and/or factors data to an AI and getting a text answer.

But the main point is that rather than just hoping the magnetisation will work the way you want it to, you can do smart recoding as if you had an assistant to work through each label. For example you can say "Relabel everything which expresses a decrease or lack of something with a ~" or "Look at all these labels and tag each with `[Food]` or `[Health]"

.

- You can even bring other columns into play, for example citation count, source count etc.
- Links recoding is significantly more powerful because you can also include the actual Quote as well as both Cause and Effect. This means the AI can make its decision with a lot more context. So this is almost like recoding from scratch, but the original coding has already identified causal claims and all we have to do is relabel the labels with the same complete information about the claim.
- Be careful: it's tempting to say things like "Find 3-8 top-level factor labels which cover the meaning of all these labels and recode them with the new top-level labels", but remember the "Rows per call" slider: with a large set of links and lots of quotes you will probably have to break up your work into multiple chunks, and each call may come up with different labels. In this case you could use the Answers mode (or the Cluster part of Soft Recode filter) first to develop some labels.

Soft recode: a useful contrast

Soft

See also

- Bulk relabelling factors for the manual routes (search/replace, Bulk Edit) in detail.
- Translate factor labels for a worked AI Answers / Factors example using a `french` Label Set.
- Recoding labels temporarily for safe experimentation with the Label Set widget.